Randomised Search Trees: The ‘Treap’

Group 35 – Aleksandr Ivanov, Martin McSweeney, Samuel Boyle, Sanidhya Arora

Aim and Background

- Built Treap in Java as a randomised balanced binary search tree
- Analysed time complexity of core data structure operations
- Performed empirical benchmarking of runtime performance
- Compared Treap’s performance with AVLTreeMap and Java’s TreeMap
- Evaluated Treap-based sorting against standard

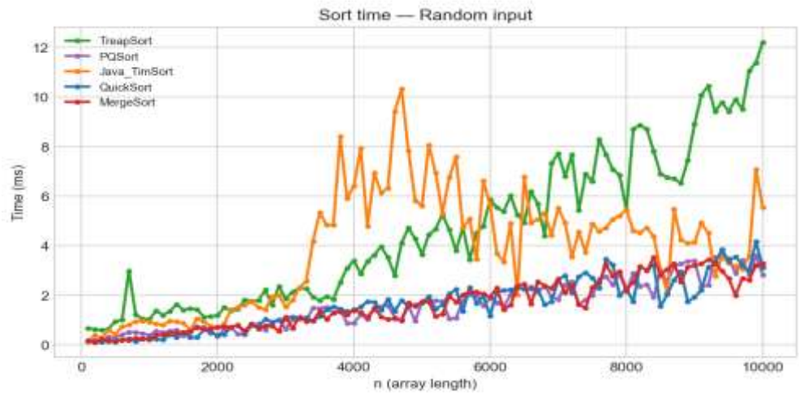
‘Treap’ Explanation

- A Treap is a hybrid of a Binary Search Tree (BST) and a Heap
- It maintains two ordering rules simultaneously:
  - BST property: nodes are ordered by key values
  - Heap property: nodes are ordered by priority values
- Each node contains:
  - Key: value being inserted
  - Priority: randomly generated value assigned at insertion
- In our implementation, the structure is:
  - Ordered by keys (BST structure)
  - Balanced using random priorities (heap structure)
- Structural rules:
  - Parent nodes always have higher priority than their children
  - Left subtree contains smaller keys, right subtree contains larger keys
- The random priority system is important because:
  - It ensures operations like search, insert, and delete run in O(log n) average time

Discussion & Key Findings

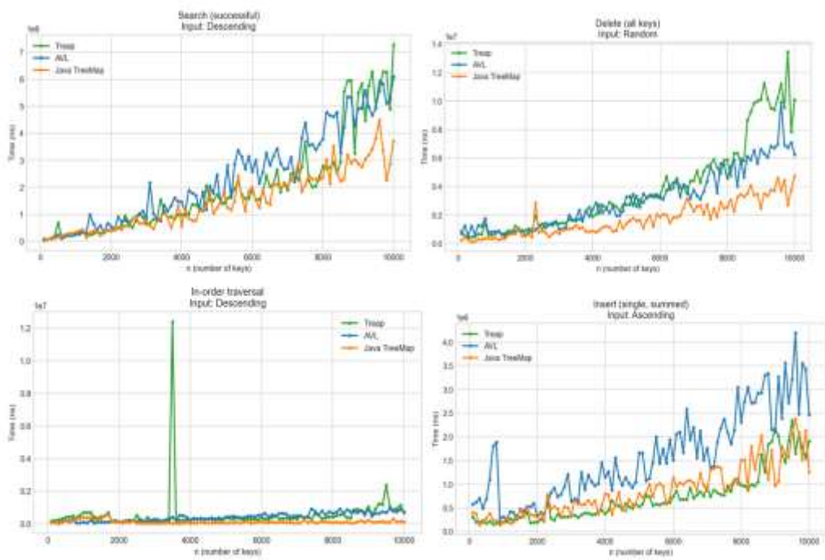
Sorting Algorithms comparisons:

- Most algorithms-maintained O(nlogn)
- TreapSort was the worst. Others clustered with larger input.



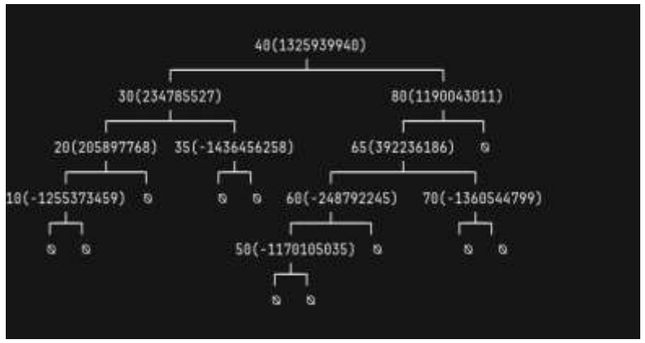
Performance Comparisons:

- All three data structures demonstrate the theoretical $O(\log\ n)$.
- Treap has inconsistencies due to it’s priority function.



Experiment Setup & Design Options

- Map performance: n=100–10k, compare Treap/AVL/TreeMap across input patterns; measure insert, get, search and delete.
- Sorting: Same n range; compare TreapSort, PQ, TimSort, quicksort, mergesort.
- Correctness: Verify sorted output after each run before accepting timing results.
- Stability check: Repeat runs with fixed seed to ensure consistent and reproducible measurements

Conclusion

- Balanced maps: All stay ~O(n log n); TreeMap is usually fastest, while treap and AVL vary by pattern but remain stable.
- Treaps: Random priorities keep expected log height, avoiding worst-case BST behaviour without storing balance factors.
- Sorting: TreapSort is slower overall; TimSort dominates on reverse-sorted data, while quicksort performs poorly.
- Conclusion: Performance reflects both JVM effects and implementation overhead, but Treaps consistently avoid worst-case behaviour, mainly because of their priority attribute.